Chess Neural Network

Caleb Faucher

University of Nebraska at Omaha

CSCI8598

Dr. Zhong

May 1st, 2026

Artificial intelligence has become one of the most influential areas of computer science in recent decades. Among the many challenges explored within artificial intelligence, chess has historically served as one of the most important benchmarks for measuring progress in machine reasoning and strategic decision-making. Chess is considered a difficult problem because the game contains an enormous number of possible board positions and legal move combinations. A successful chess-playing system must be able to evaluate positions, anticipate future consequences of moves, and develop long-term strategies while reacting to the opponent's actions. Because of these characteristics, chess provides an ideal environment for studying machine learning and intelligent decision-making systems.

The purpose of this project was to investigate how deep learning can be applied to the game of chess and how neural networks can be combined with search algorithms to create an intelligent agent capable of learning through self-play. The central objective of this work was not simply to create a program that follows predefined rules or memorized strategies, but rather to design a system that learns from experience. Traditional chess engines often rely on handcrafted evaluation functions developed by expert programmers. These evaluation functions assign scores to board positions using manually designed heuristics such as material advantage, king safety, pawn structure, and positional control. While these methods have historically produced very strong chess engines, they require extensive domain knowledge and significant human effort.

Recent developments in artificial intelligence have shown that neural networks can learn many of these concepts automatically through training. Instead of programming strategic principles directly into the system, researchers have demonstrated that deep neural networks can discover useful patterns by repeatedly playing games and adjusting internal parameters based on outcomes. Inspired by these developments, this project explores whether a simplified version of such a system can be implemented using accessible tools and limited computational resources.

The primary problem addressed in this project is how to enable a machine learning model to learn effective chess strategies through reinforcement and self-play while also making intelligent move decisions during gameplay. A second challenge involves integrating a search algorithm capable of exploring future positions rather than relying entirely on the neural network's immediate predictions. Without search, a neural network may fail to recognize tactical consequences several moves ahead. Therefore, the project investigates how Monte Carlo Tree Search can improve decision quality by allowing the model to simulate future possibilities before selecting a move.

The successful implementation of such a system has applications beyond chess itself. The ability to combine learning and planning is relevant in many real-world domains involving sequential decision-making. Autonomous robotics systems often need to plan actions while adapting to changing environments. Logistics and supply chain optimization problems require systems that can evaluate multiple future scenarios and choose efficient strategies. Financial forecasting and resource allocation also involve long-term planning under uncertainty. Therefore, while chess serves as the experimental environment in this project, the underlying principles extend to broader artificial intelligence applications.

The history of artificial intelligence in chess dates back many decades and reflects the evolution of computational methods over time. Early chess programs relied primarily on brute-force search

techniques. These systems generated large numbers of possible future moves and evaluated resulting positions using manually designed scoring systems. One of the most influential algorithms in classical chess engines is the minimax algorithm combined with alpha-beta pruning. Minimax attempts to maximize the player's advantage while minimizing the opponent's opportunities, whereas alpha-beta pruning improves efficiency by eliminating branches of the search tree that are unlikely to influence the final decision.

Modern engines such as Stockfish represent the culmination of this classical approach. Stockfish achieves extremely high playing strength by combining deep search techniques with carefully engineered evaluation functions and optimization methods. However, these engines still rely heavily on handcrafted knowledge provided by expert programmers. Their strength comes not from learning independently but from the quality of the search algorithm and evaluation heuristics designed by humans.

A major transformation occurred with the development of deep reinforcement learning systems such as AlphaZero. AlphaZero demonstrated that a machine learning system could achieve superhuman chess performance without access to human opening databases or handcrafted evaluation functions. Instead, the system learned entirely through self-play. AlphaZero used a deep neural network that produced both policy and value predictions. The policy component estimated the probability distribution over legal moves, while the value component estimated the likelihood of winning from a given position. These outputs were combined with Monte Carlo Tree Search, allowing the system to balance exploration of new moves with exploitation of moves predicted to be strong.

The success of AlphaZero significantly influenced research in reinforcement learning and game-playing AI. It showed that systems could develop advanced strategic understanding without explicit human guidance. However, reproducing a full-scale AlphaZero implementation requires enormous computational resources, including large GPU clusters and millions of self-play games. For educational and experimental purposes, smaller-scale implementations provide valuable opportunities to understand the underlying principles without requiring industrial-scale hardware.

The approach implemented in this project follows the same general philosophy as AlphaZero but in a simplified and computationally accessible form. The model uses a smaller convolutional neural network implemented in PyTorch and trains using a limited number of self-play games. Monte Carlo Tree Search is incorporated to improve move selection and provide planning capability. Although the resulting model is far weaker than professional chess engines, it demonstrates many of the same fundamental ideas and behaviors.

The methodology of this project consists of several interconnected components, including board representation, neural network architecture, move encoding, self-play training, and Monte Carlo Tree Search integration. Each component contributes to the overall learning and decision-making process.

The first major design decision involves how to represent the chessboard as input to the neural network. Neural networks operate on numerical tensors, so the symbolic information of chess

pieces must be converted into a structured numerical format. In this implementation, the board is represented as a tensor of dimensions:

12*8*812 * 8 *81**8*8

The twelve channels correspond to the six types of chess pieces for each player. One channel represents white pawns, another represents white knights, and so on, while additional channels represent the corresponding black pieces. Each square on the chessboard is encoded with binary values indicating whether a particular piece occupies that location. This representation preserves the spatial structure of the board and allows convolutional neural networks to identify patterns involving piece relationships and positional arrangements.

The neural network architecture itself consists of convolutional layers followed by fully connected layers. Convolutional neural networks are particularly well suited for grid-based data because they can detect local spatial patterns. In chess, local interactions between pieces are extremely important. For example, the relationship between a king and nearby attacking pieces may determine whether the king is in danger. Similarly, pawn structures often involve neighboring squares and local positional configurations. By applying convolutional filters across the board representation, the network can learn to identify meaningful tactical and positional features.

The network begins with convolutional layers that extract increasingly abstract representations of the board. Rectified Linear Unit activation functions are used after each convolution to introduce nonlinearity and improve learning capacity. After feature extraction, the resulting representations are flattened and passed through fully connected layers that produce two outputs. The first output is the policy head, which predicts move probabilities. The second output is the value head, which predicts the expected outcome of the game.

The value output is constrained to the range between negative one and positive one using a hyperbolic tangent activation function. A value of positive one indicates a predicted win, negative one indicates a predicted loss, and zero indicates a draw or balanced position. This value estimate serves as the network's evaluation of the current board state.

The move encoding system maps legal chess moves into numerical indices. Each move is represented using its starting and ending squares. Because the chessboard contains sixty-four squares, the total number of possible square-to-square combinations equals:

64*64=409664 * 64 = 409664*64=4096

Therefore, the policy head outputs a vector containing probabilities for all possible move indices. During gameplay, only legal moves are considered, and illegal moves are ignored.

Although the neural network provides move probabilities and position evaluations, relying solely on these predictions often produces weak gameplay. Neural networks may fail to recognize tactical traps or long-term consequences several moves ahead. To address this limitation, the project integrates Monte Carlo Tree Search.

Monte Carlo Tree Search is a search algorithm that incrementally builds a tree of explored positions. The algorithm repeatedly performs four stages: selection, expansion, evaluation, and backpropagation. During selection, the algorithm traverses the existing search tree by choosing moves according to a balance between exploration and exploitation. This balance is determined using the following PUCT equation:

$$U(s,a) = Q(s,a) + c_{puct} P(s,a) \frac{\sqrt{N(s)}}{1+N(s,a)}$$

In this equation, Q(s,a)Q(s,a)Q(s,a) represents the estimated quality of taking action aaa in state sss, while P(s,a)P(s,a)P(s,a) represents the neural network's prior probability for that move. The term involving visit counts encourages exploration of less-visited nodes while still favoring moves believed to be promising.

After selection, the algorithm expands the tree by adding unexplored legal moves. The neural network then evaluates the resulting positions and produces value estimates. Finally, the value estimates are propagated backward through the visited nodes, updating their statistics. Repeating this process many times allows the search tree to focus increasingly on strong candidate moves.

Training is performed entirely through self-play using the python-chess framework. During self-play, the model competes against itself repeatedly, generating training examples automatically. Each game produces sequences of board states, selected moves, and final outcomes. These examples are stored and used to train the neural network.

The loss function combines two components. The policy loss measures the difference between predicted move probabilities and selected moves using cross-entropy loss. The value loss measures the difference between predicted game outcomes and actual outcomes using mean squared error. The total loss is the sum of these two components, encouraging the network to improve both move prediction and board evaluation simultaneously.

Several experiments were conducted to evaluate the system's behavior and learning progress. Initial experiments involved testing the neural network without Monte Carlo Tree Search. In these experiments, move selection relied entirely on the network's policy output. The resulting gameplay appeared highly random, and the system frequently made obvious tactical mistakes. This outcome was expected because the neural network had limited training data and no ability to analyze future consequences.

After integrating Monte Carlo Tree Search, the quality of gameplay improved noticeably. The system began to prioritize captures, avoid immediate losses of material, and occasionally execute basic tactical sequences. The search algorithm compensated for weaknesses in the neural network by exploring future positions before making decisions. Although the model remained relatively weak compared to professional engines, the improvement demonstrated the importance of combining learning with search.

Additional experiments examined the effects of training duration. Over successive epochs, the training loss gradually decreased, indicating that the network was learning useful patterns from self-play data. Early training epochs produced highly unstable gameplay with frequent blunders. Later epochs showed more coherent piece development and improved positional awareness. The model occasionally demonstrated basic opening principles such as controlling the center and developing minor pieces.

Human-versus-model testing provided some of the most informative observations throughout the development process because it allowed direct comparison between the model's behavior and human gameplay. As the developer of the system, I also served as one of the primary test players during experimentation. My own chess experience can best be described as intermediate level. Before beginning this project, I had experience playing casual online chess and possessed a basic understanding of common opening principles, tactical patterns, and endgame concepts. However, I was not an advanced tournament-level player and often relied more on general positional understanding than deep calculation. This made the testing process especially interesting because the model's progression could be observed from the perspective of an average human player rather than a professional chess expert.

During the earliest stages of development, the model performed extremely poorly in direct games against me. Before Monte Carlo Tree Search was implemented, the neural network selected moves based almost entirely on untrained or weakly trained policy predictions. As a result, the model frequently made nonsensical moves, ignored direct threats, and sacrificed important pieces without compensation. In many games, the model would move the same piece repeatedly in the opening or fail to respond to simple tactical attacks. Checkmates could often be achieved within only a few moves because the system lacked both tactical awareness and positional understanding. At this stage, playing against the model felt less like playing against a chess opponent and more like playing against a random move generator with occasional legal move selection.

After additional self-play training epochs were completed, slight improvements became noticeable. The model began demonstrating a limited understanding of material preservation and legal positional development. Although many mistakes still occurred, the games became somewhat more structured. The model occasionally developed knights and bishops toward the center of the board and showed basic recognition of captures. During this phase, I observed that the system could sometimes punish careless mistakes if I played too aggressively or ignored tactical opportunities. However, victories against the model were still relatively easy to achieve because it lacked long-term planning and often collapsed after losing material.

The largest improvement occurred after Monte Carlo Tree Search was integrated into the system. The addition of search dramatically changed the quality of gameplay because the model was no longer relying entirely on immediate neural network predictions. Instead, it began evaluating future positions before making decisions. During matches played after the MCTS implementation, I noticed that the model became significantly more resistant to simple tactical tricks. It no longer blundered pieces as frequently and occasionally created threats that required careful responses. In several games, the model successfully defended against direct attacks that earlier versions would have failed to recognize.

One particularly noticeable improvement involved the model's opening play. Earlier versions often violated basic opening principles, while the MCTS-enhanced version more consistently developed pieces and attempted to control central squares. Although the openings were still far from optimal compared to professional engines, they resembled legitimate chess games much more closely than before. This made the matches feel increasingly competitive and realistic.

As training continued, the model also began exhibiting signs of tactical awareness. For example, there were situations in which the system recognized opportunities for forks, discovered attacks, or exchanges that improved its position. While these tactical ideas were not always executed perfectly, they demonstrated that the combination of self-play and search was allowing the system to learn meaningful chess concepts. In some matches, I found myself needing to think carefully before attacking because careless moves could now result in the loss of material or positional disadvantage.

Despite these improvements, the model still displayed several weaknesses throughout testing. Strategic planning remained limited, especially in complex middle-game positions requiring long-term positional understanding. The system often struggled in endgames and occasionally made passive moves that allowed easy counterplay. Additionally, because the model had been trained on only a relatively small number of self-play games, its play style sometimes appeared inconsistent. Some games showed surprisingly competent tactical sequences, while others contained obvious mistakes that stronger chess players would rarely make.

From a personal perspective, the progression of the model throughout development was one of the most rewarding aspects of the project. Observing the transition from random move selection to partially coherent gameplay provided practical insight into how machine learning systems gradually acquire useful behaviors through training. Playing repeated games against the evolving model also made the improvements easier to recognize than numerical loss values alone. Even though the model never reached a level where it consistently challenged experienced human players, the steady increase in competitiveness demonstrated that the system was genuinely learning from self-play and benefiting from the integration of Monte Carlo Tree Search.

Despite these promising results, several limitations remain. The neural network used in this project is relatively small and lacks the complexity of modern deep reinforcement learning systems. The amount of training data generated through self-play was also limited by hardware constraints. Advanced systems often train on millions of games using specialized hardware such as GPUs or TPUs. In addition, the current implementation does not include advanced techniques such as residual networks, replay buffers, Dirichlet noise, or parallelized self-play generation.

Future work could significantly improve the system's performance. Increasing the number of training games would provide the neural network with more experience and improve generalization. Larger and deeper network architectures could capture more sophisticated positional concepts. Replay buffers could help stabilize training by allowing the model to learn from a broader distribution of previous experiences. More advanced exploration strategies could encourage the discovery of stronger moves during self-play.

In conclusion, this project demonstrates that a neural network combined with Monte Carlo Tree Search can learn to play chess at a basic level through self-play. The project successfully integrates deep learning techniques with search-based planning and illustrates many of the core ideas behind modern reinforcement learning systems. Although the resulting model is limited in strength and computational scale, it provides valuable insight into how intelligent agents can learn complex decision-making tasks without explicit human programming. The experiments conducted show that combining neural evaluation with search significantly improves gameplay quality compared to relying on the neural network alone. Overall, this work serves as both an educational exploration of artificial intelligence techniques and a foundation for future improvements in game-playing systems and reinforcement learning research.